

Lecture 4 supplement 1 Ensemble learning

**01211373 : Machine Learning and
Programming for Industry**

Dr. Varodom Toochinda

Dept. of Mechanical Engineering,
Kasetsart University



Topics

- ❑ Ensemble Methods
 - ❑ Majority voting
 - ❑ Bagging (Bootstrap Aggregating)
 - ❑ Random Forest (Bagging + Random feature selection)
 - ❑ Boosting
 - ❑ Adaboost
 - ❑ Gradient boosting
 - ❑ XGboost

Ensemble methods

From AI Overview

"An **ensemble** is a group of people, musicians, actors, or things that perform or work together to create a single complete unit."

<https://youtu.be/wVRidHbwLB0?si=hQsinkIbUX7Q7UmT>

Ensemble approach based on majority voting

$$\hat{y} = \arg \max_i \sum_{j=1}^m w_j \chi_A(C_j(x) = i)$$

$$\hat{y} = \text{mode}(C_1(x), C_2(x), \dots, C_m(x))$$

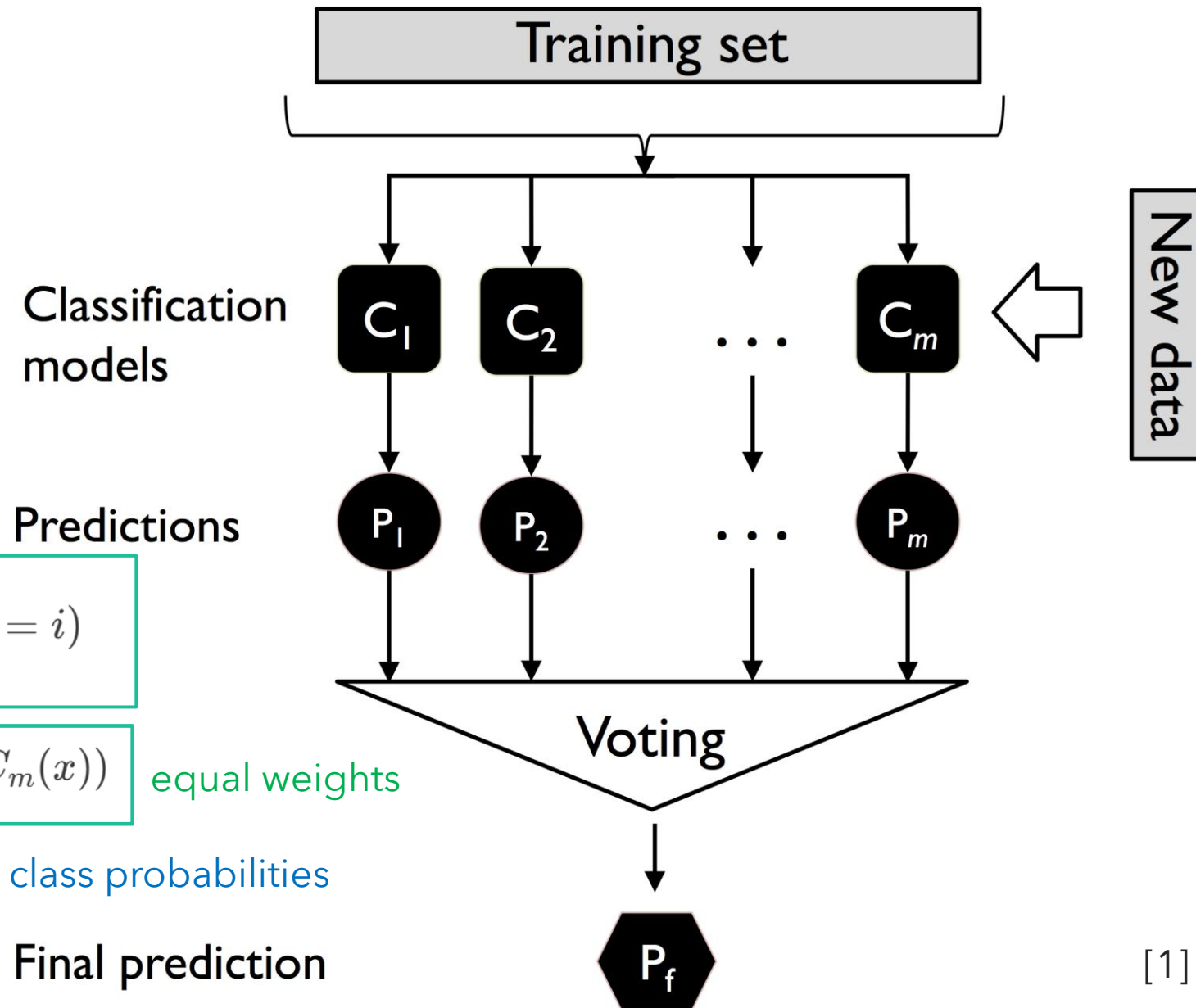
equal weights

$$\hat{y} = \arg \max_i \sum_{j=1}^m w_j p_{ij}$$

class probabilities

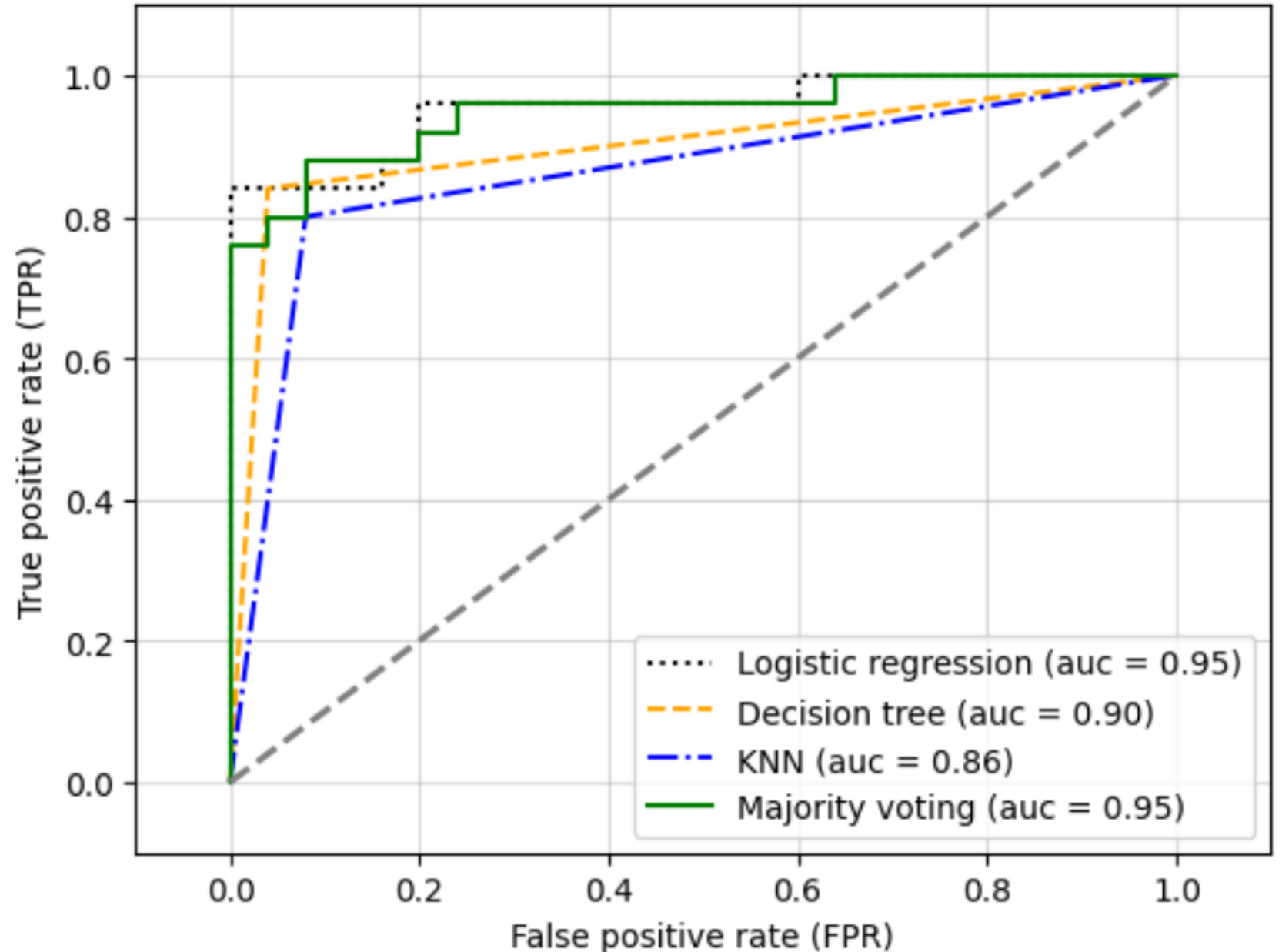
Final prediction

"Soft" Voting

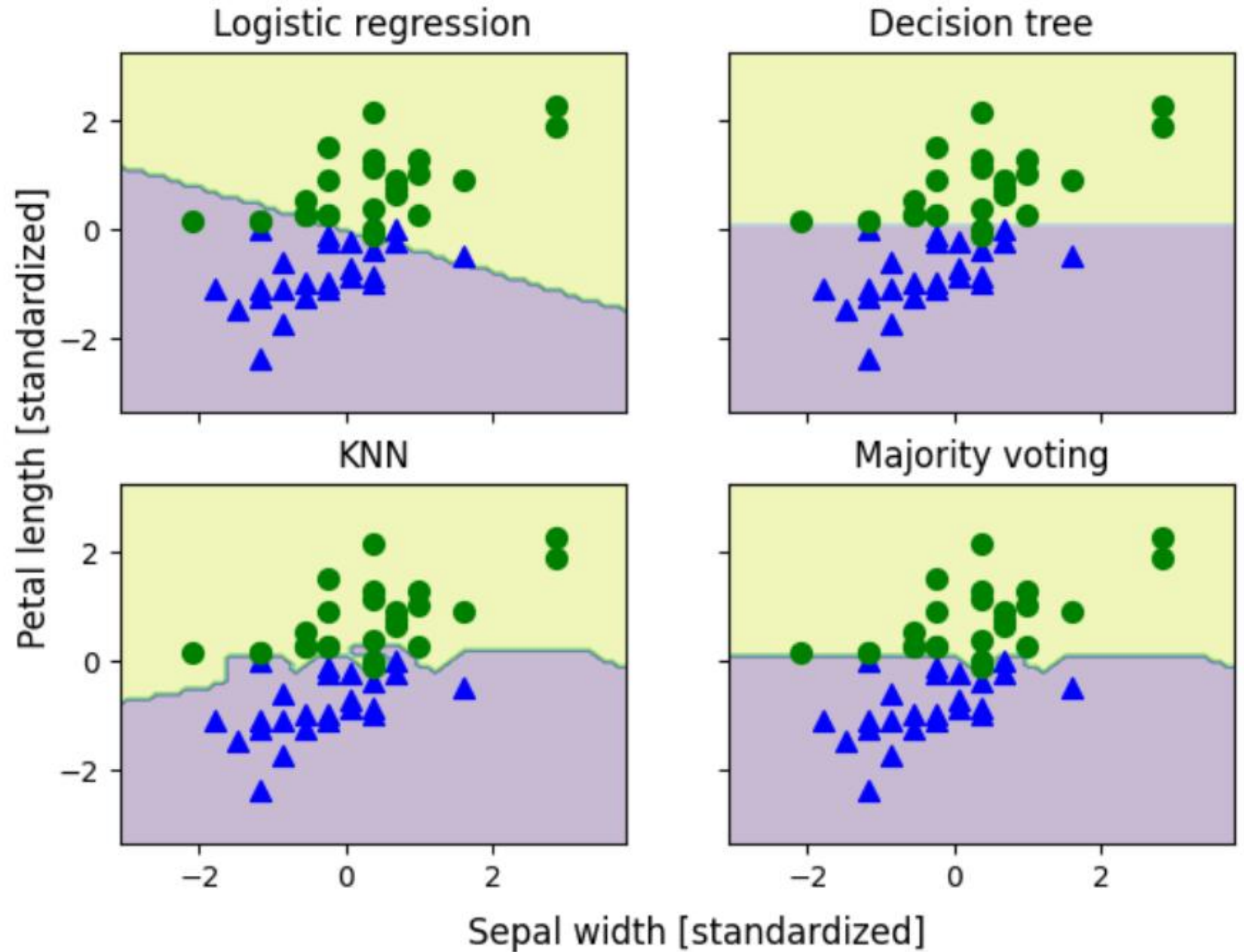


[1]

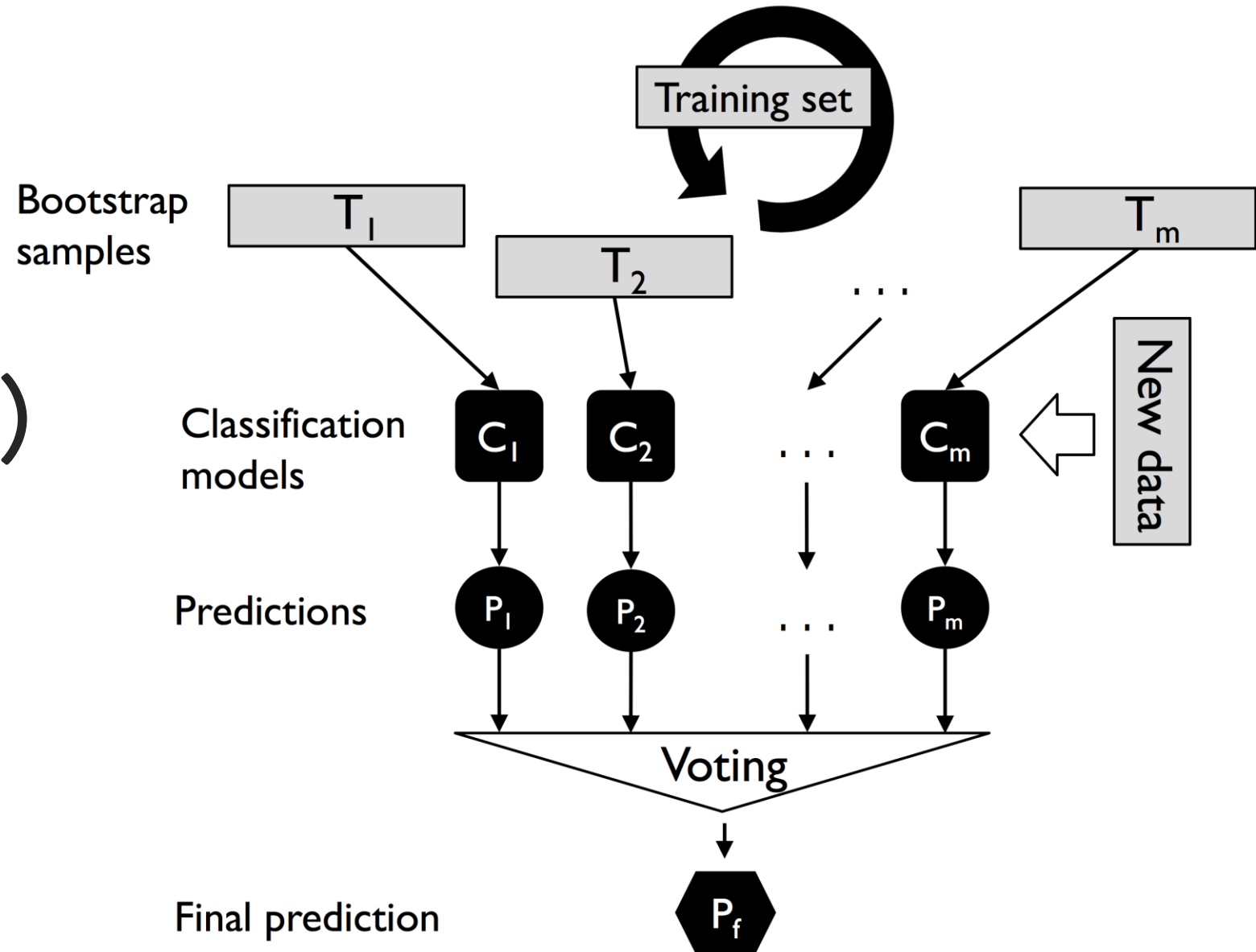
Evaluating and tuning ensemble classifier



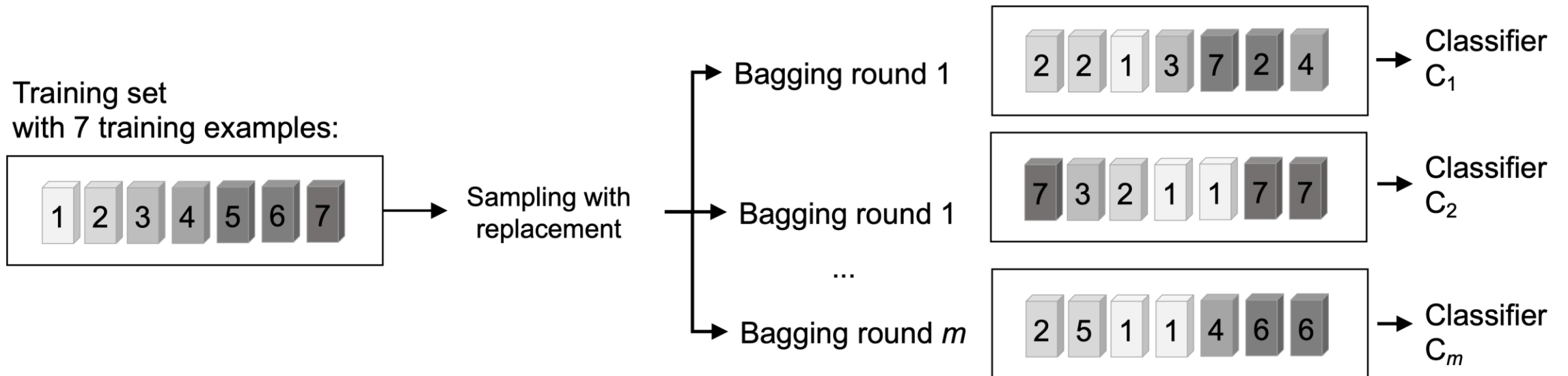
Decision boundaries for the different classifiers



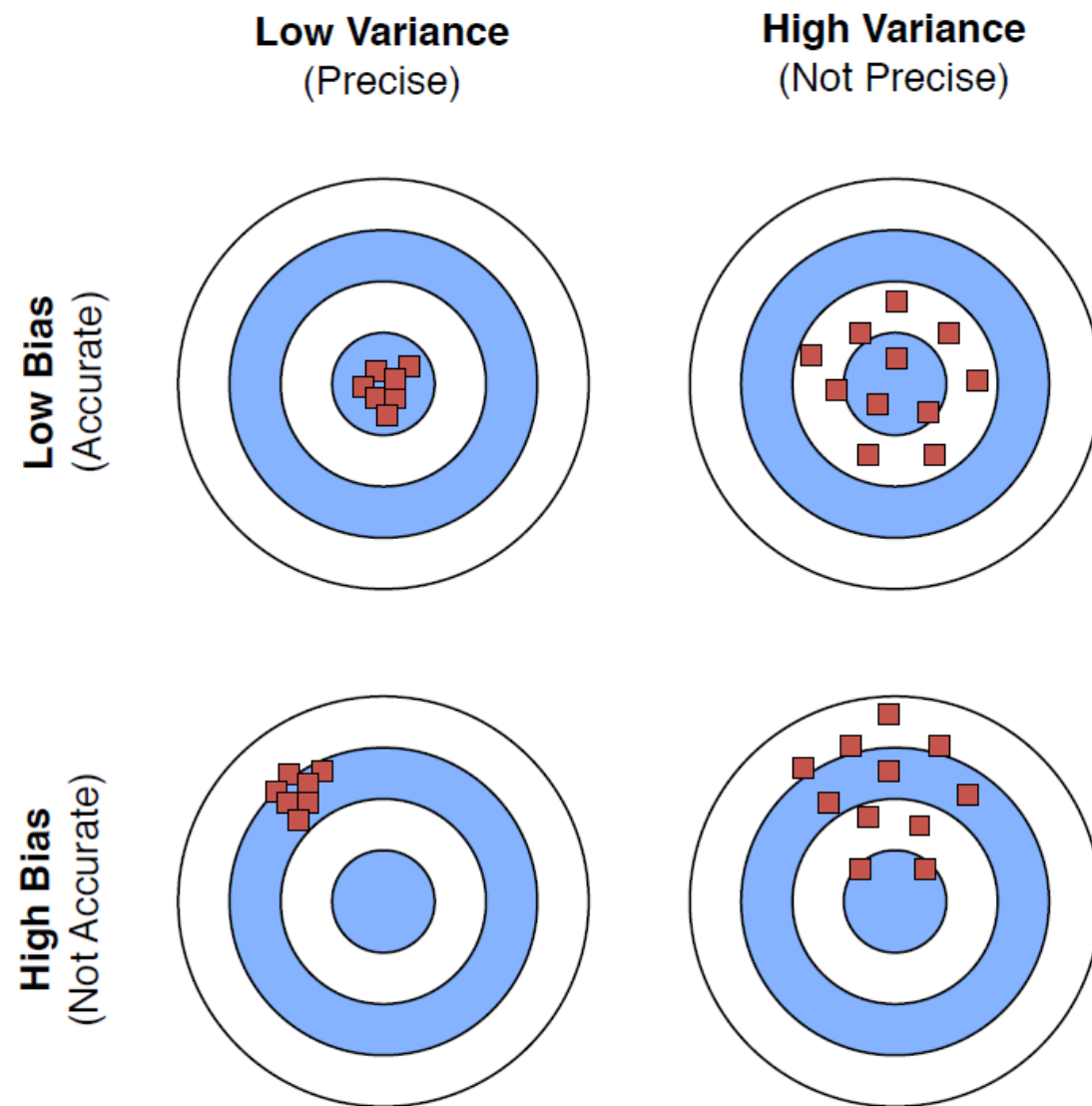
Bagging (bootstrap aggregating)



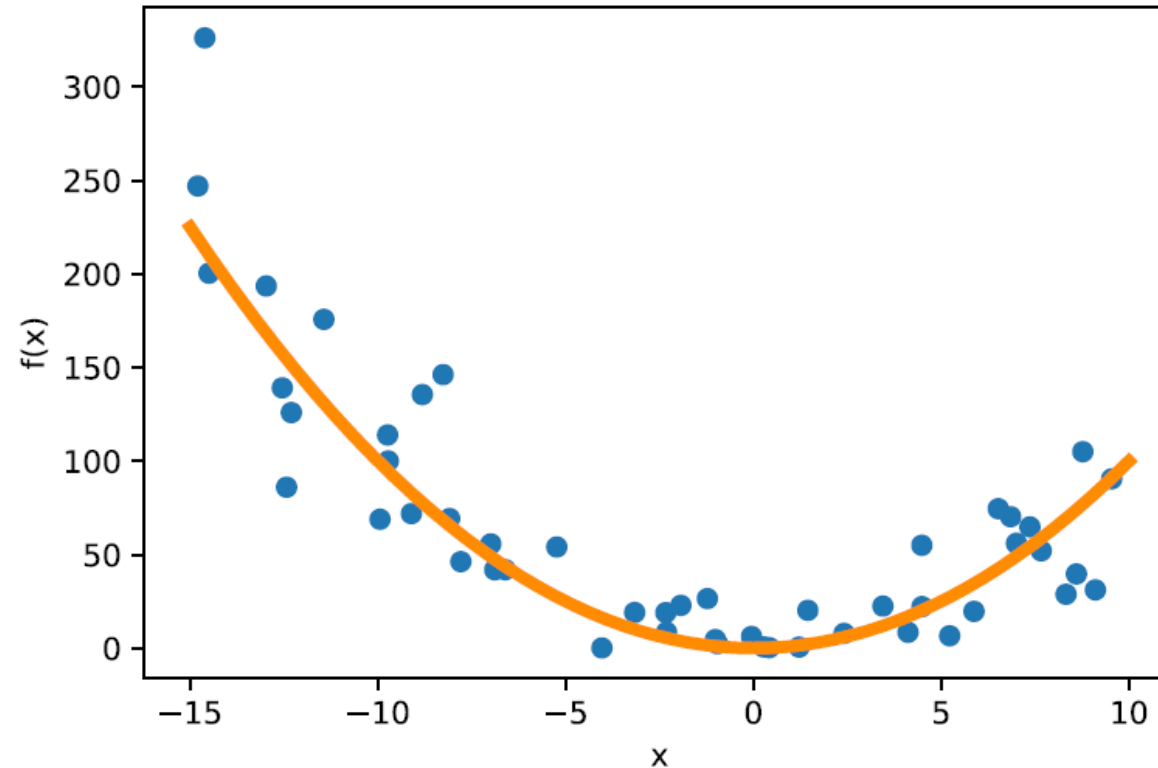
Bagging example



Bias-Variance Intuition



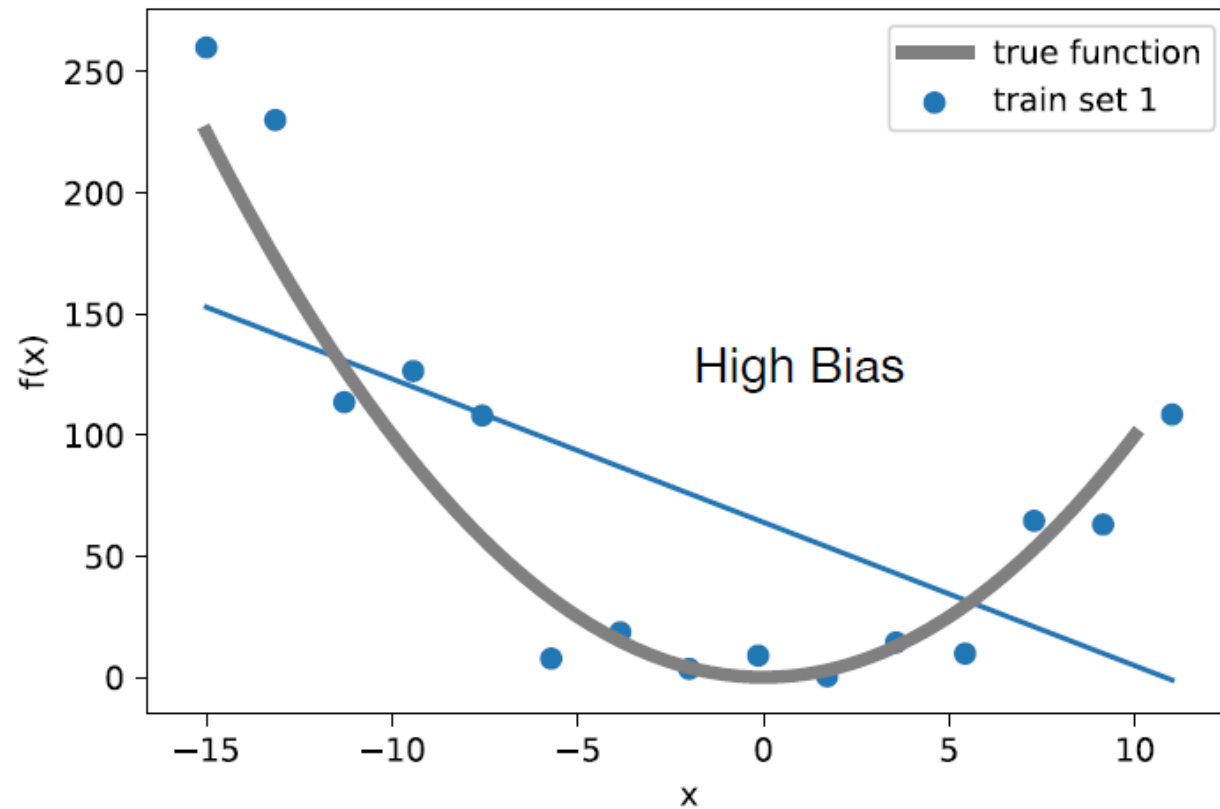
Bias and Variance Example



where $f(x)$ is some true (target) function

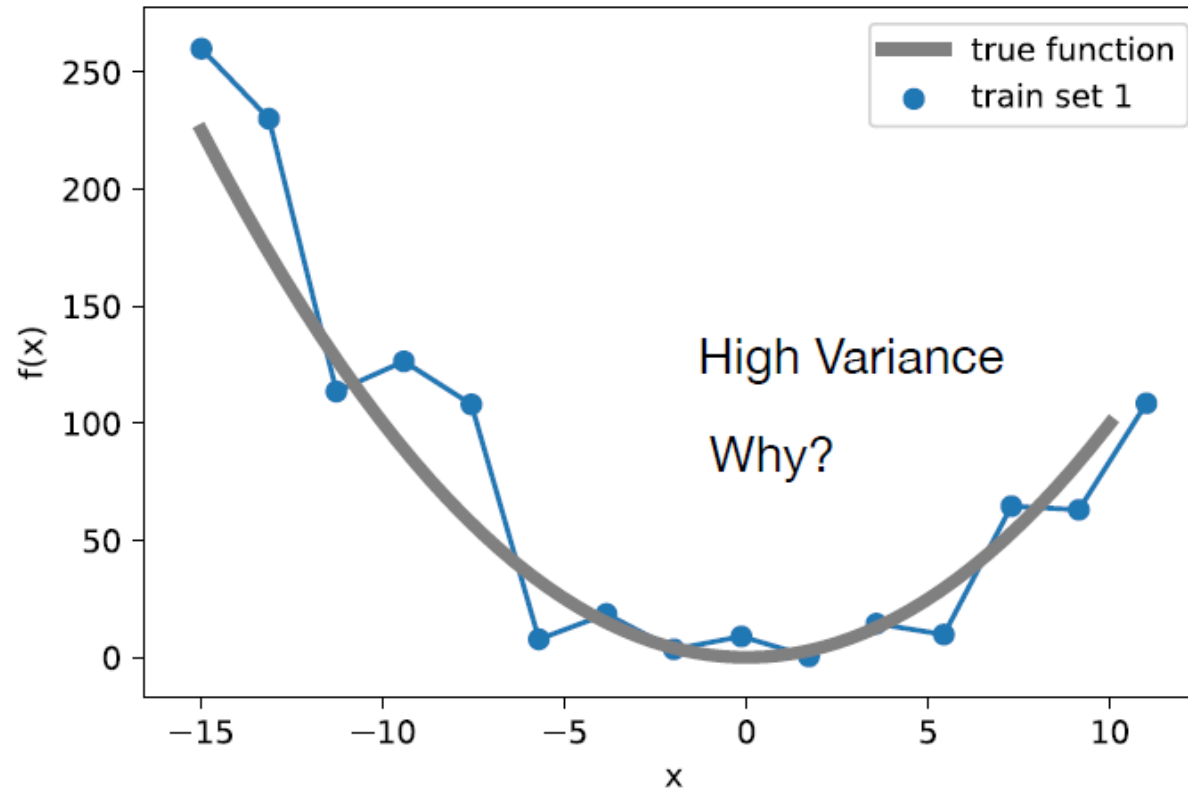
the blue dots are a training dataset;
here, I added some random Gaussian noise

Bias and Variance Example



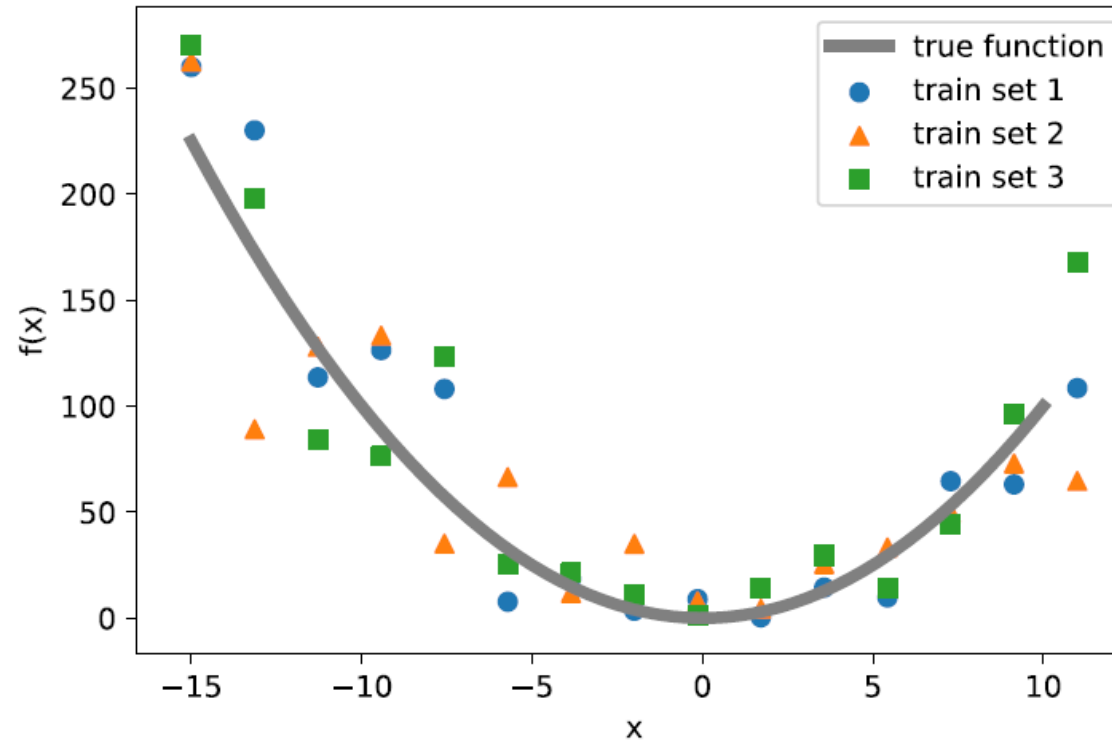
here, suppose I fit a simple linear model (linear regression)
or a decision tree stump

Bias and Variance Example



here, suppose I fit an unpruned decision tree

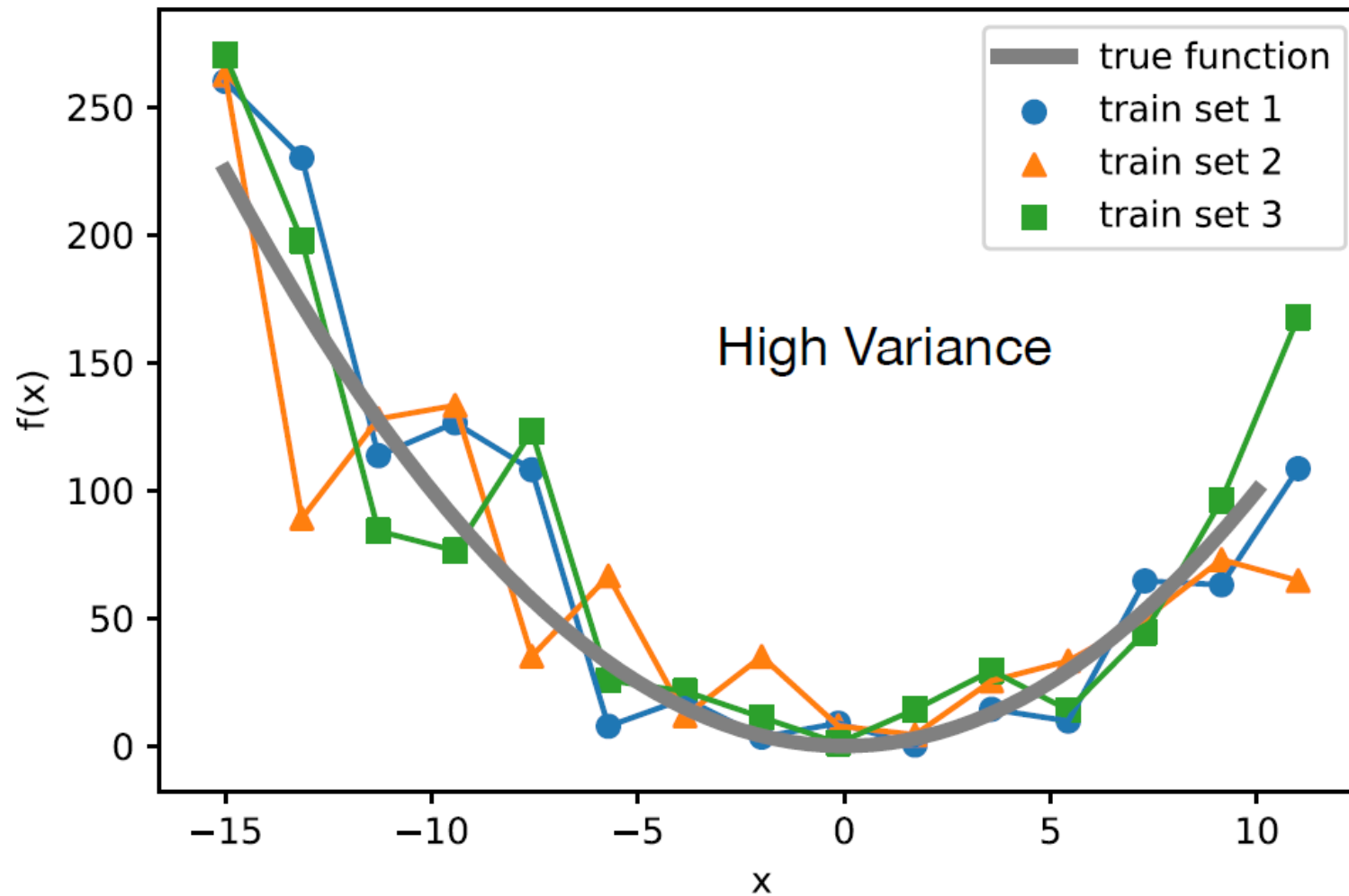
Bias and Variance Example



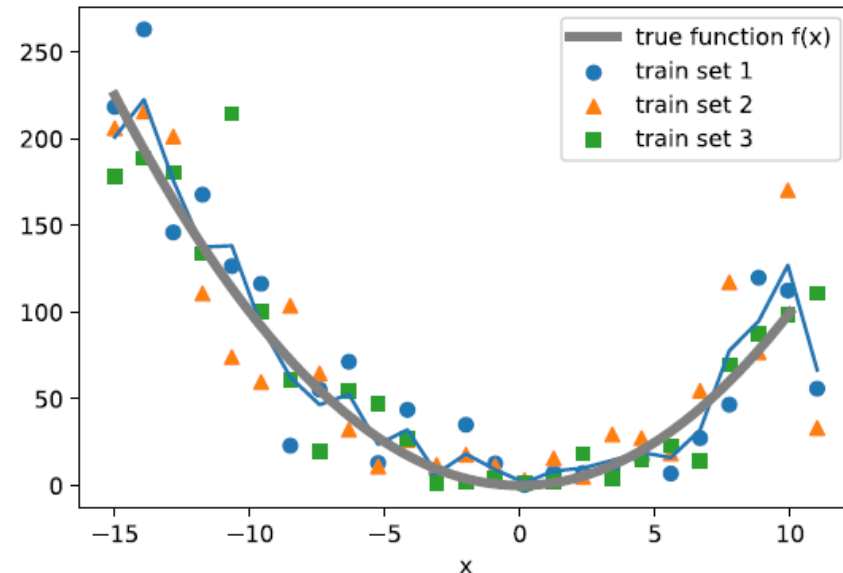
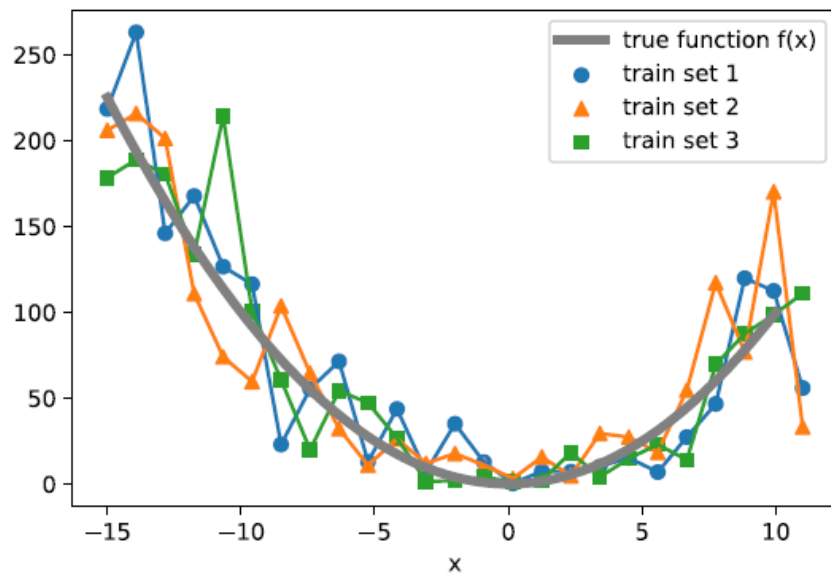
where $f(x)$ is some true (target) function

suppose we have multiple training sets

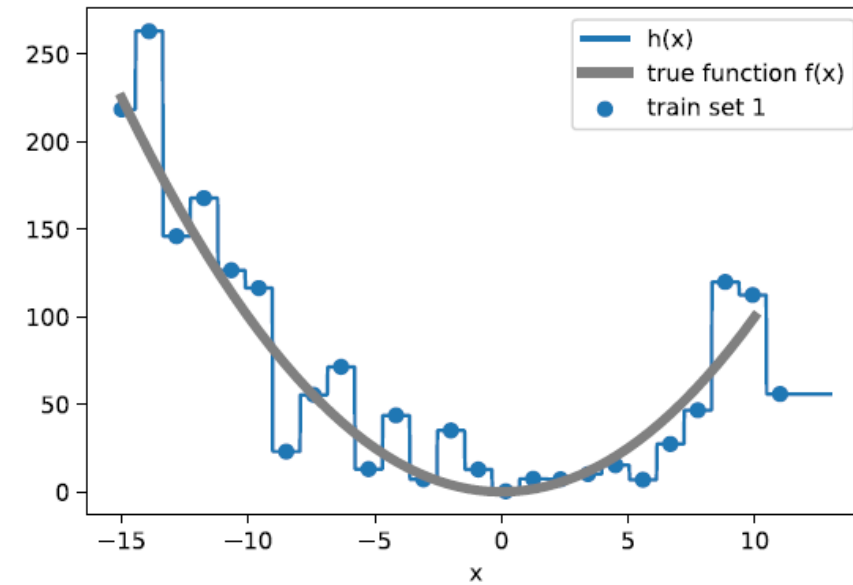
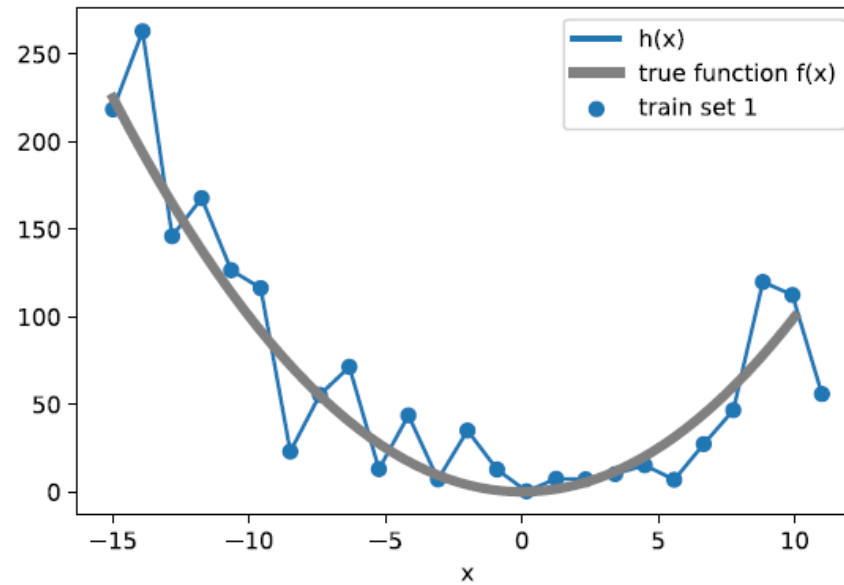
Bias and Variance Example



Clarifications 1



Clarifications 2



```
from sklearn import model_selection
from sklearn.tree import DecisionTreeClassifier
from sklearn.model_selection import train_test_split
from sklearn import datasets
from sklearn.ensemble import BaggingClassifier

iris = datasets.load_iris()
X, y = iris.data[:, [0, 3]], iris.target

X_train, X_test, y_train, y_test = \
    train_test_split(X, y, test_size=0.25, random_state=1)

X_train, X_val, y_train, y_val = \
    train_test_split(X_train, y_train, test_size=0.25, random_state=1)

print('Train/Valid/Test sizes:', y_train.shape[0], y_val.shape[0], y_test.shape[0])

tree = DecisionTreeClassifier(criterion='entropy',
                             random_state=1,
                             max_depth=None)

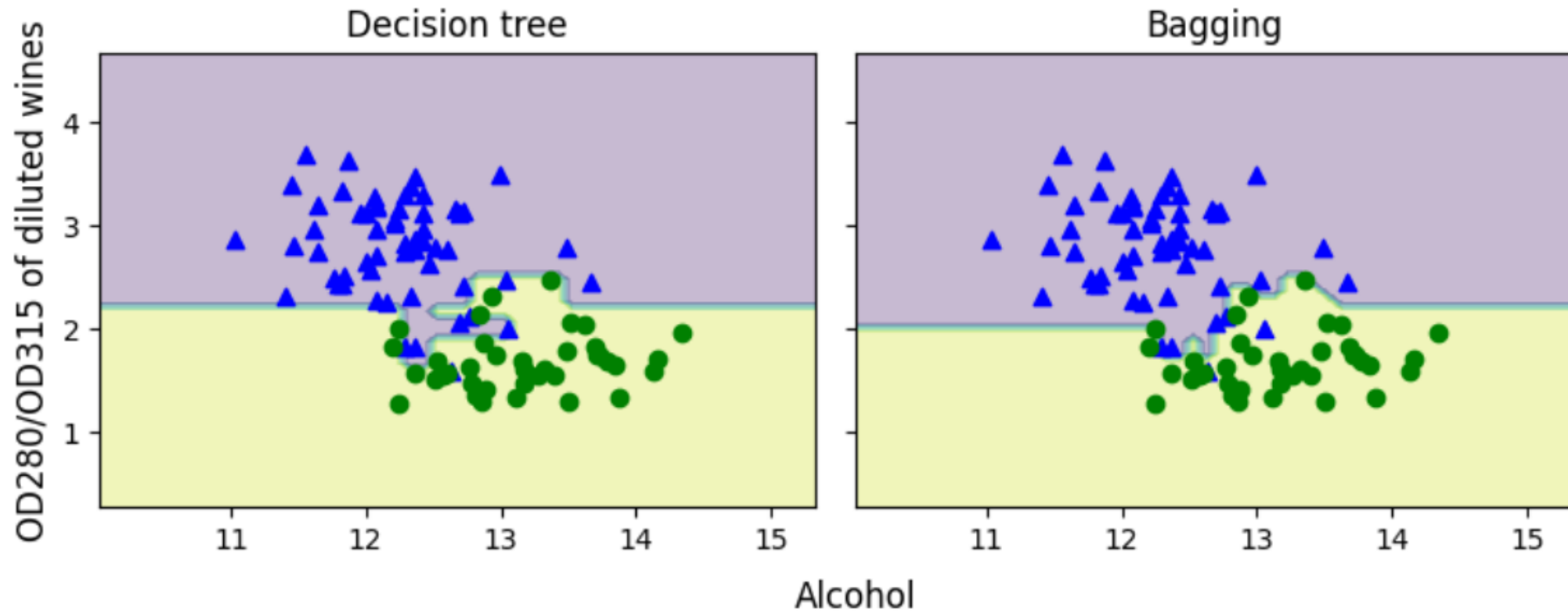
bag = BaggingClassifier(base_estimator=tree,
                        n_estimators=500,
                        oob_score=True,
                        bootstrap=True,
                        bootstrap_features=False,
                        n_jobs=1,
                        random_state=1)

bag.fit(X_train, y_train)

print("OOB Accuracy: %0.2f" % bag.oob_score_)
print("Test Accuracy: %0.2f" % bag.score(X_test, y_test))

Train/Valid/Test sizes: 84 28 38
OOB Accuracy: 0.93
Test Accuracy: 0.95
```

Decision regions for wine dataset



Boosting

The background is a solid teal color. It features several large, white, organic, blob-like shapes that overlap each other. There are also four white circles of different sizes scattered across the teal background: one in the top-left, one in the top-right, one in the bottom-right, and one in the bottom-left.

Adaptive Boosting Less used nowadays

e.g., AdaBoost

Freund, Y., & Schapire, R. E. (1997). A decision-theoretic generalization of on-line learning and an application to boosting. *Journal of computer and system sciences*, 55(1), 119-139.

Differ mainly in terms of how

- weights are updated
- classifiers are combined

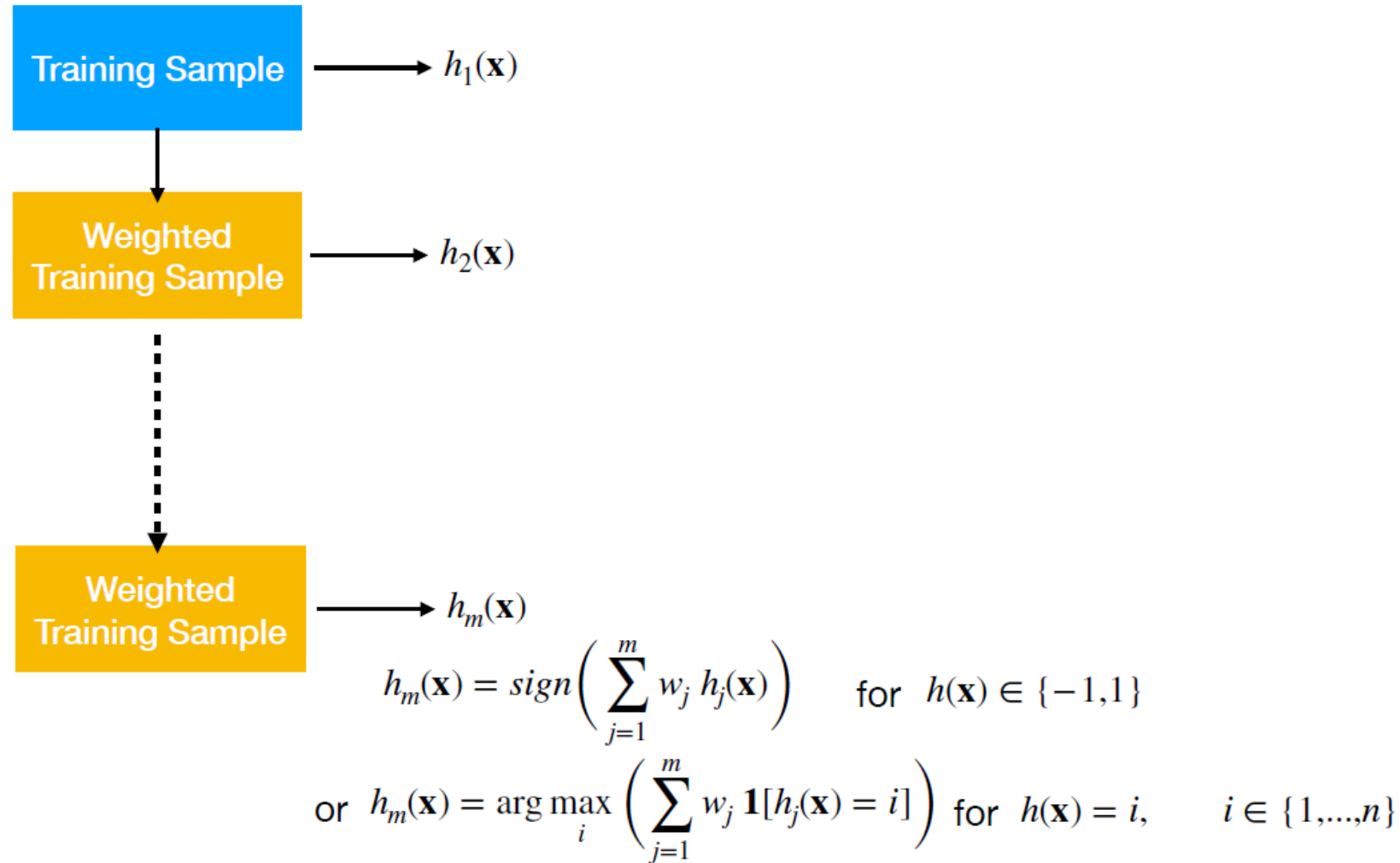
Gradient Boosting

e.g., LightGBM, XGBoost, scikit-learn's GradientBoostingClassifier

Friedman, J. H. (2001). Greedy function approximation: a gradient boosting machine. *Annals of statistics*, 1189-1232.

Chen, T., & Guestrin, C. (2016). Xgboost: A scalable tree boosting system. In *Proceedings of the 22nd ACM SIGKDD International conference on knowledge discovery and data mining* (pp. 785-794). ACM.

General Boosting



General Boosting

- ▶ Initialize a weight vector with uniform weights
- ▶ Loop:
 - ▶ Apply weak learner* to weighted training examples (instead of orig. training set, may draw bootstrap samples with weighted probability)
 - ▶ Increase weight for misclassified examples
- ▶ (Weighted) majority voting on trained classifiers

<https://youtu.be/LxcGKNV5-p4?si=ldoCBbvPuBVnxSjK>

* a learner slightly better than random guessing

AdaBoost

Less used nowadays, so omitted. See [1],[4].

Gradient Boosting

<https://youtu.be/zblsrxc7XpM?si=lpT3-EN4Qo-d-uMI>

Gradient Boosting

Gradient boosting is somewhat similar to AdaBoost:

- trees are fit sequentially to improve error of previous trees
- boost weak learners to a strong learner

The way how the trees are fit sequentially differs in AdaBoost and Gradient Boosting, though ...

Friedman, J. H. (1999). "Greedy Function Approximation: A Gradient Boosting Machine".

Gradient Boosting -- Conceptual Overview

- **Step 1:** Construct a base tree (just the root node)
- **Step 2:** Build next tree based on errors of the previous tree
- **Step 3:** Combine tree from step 1 with trees from step 2. Go back to step 2.

Gradient Boosting -- Conceptual Overview

--> A Regression-based Example

x1# Rooms	x2=City	x3=Age	y=Price
5	Boston	30	1.5
10	Madison	20	0.5
6	Lansing	20	0.25
5	Waunakee	10	0.1

In million US Dollars

- **Step 1:** Construct a base tree (just the root node)

$$\hat{y}_1 = \frac{1}{n} \sum_{i=1}^n y^{(i)} = 0.5875$$

Gradient Boosting -- Conceptual Overview

--> A Regression-based Example

- **Step 2:** Build next tree based on errors of the previous tree

First, compute (pseudo) residuals: $r_1 = y_1 - \hat{y}_1$

In million US Dollars

x1#	x2=City	x3=Age	y=Price	r1=Res
5	Boston	30	1.5	$1.5 - 0.5875 = 0.9125$
10	Madison	20	0.5	$0.5 - 0.5875 = -0.0875$
6	Lansing	20	0.25	$0.25 - 0.5875 = -0.3375$
5	Waunake	10	0.1	$0.1 - 0.5875 = -0.4875$

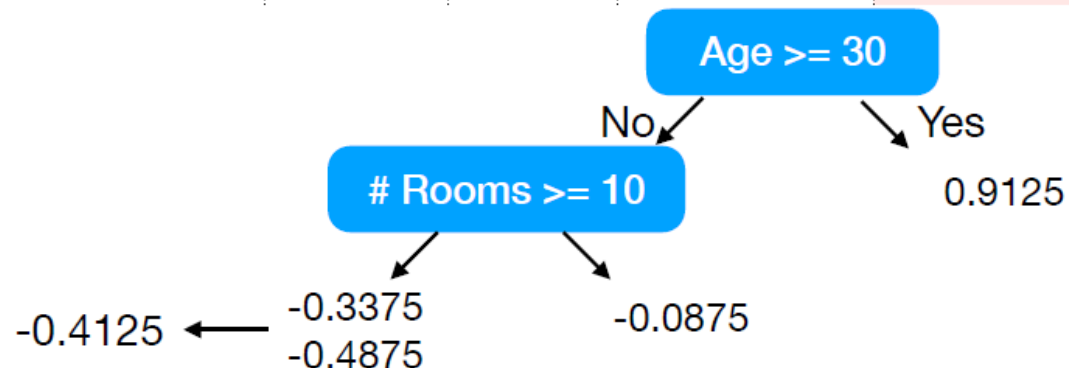
Gradient Boosting -- Conceptual Overview

--> A Regression-based Example

- **Step 2:** Build next tree based on errors of the previous tree

Then, create a tree based on $x_1, \dots, x_m$ to fit the residuals

x1#	x2=City	x3=Age	y=Price	r1=Residual
5	Boston	30	1.5	$1.5 - 0.5875 = 0.9125$
10	Madison	20	0.5	$0.5 - 0.5875 = -0.0875$
6	Lansing	20	0.25	$0.25 - 0.5875 = -0.3375$
5	Waunake	10	0.1	$0.1 - 0.5875 = -0.4875$

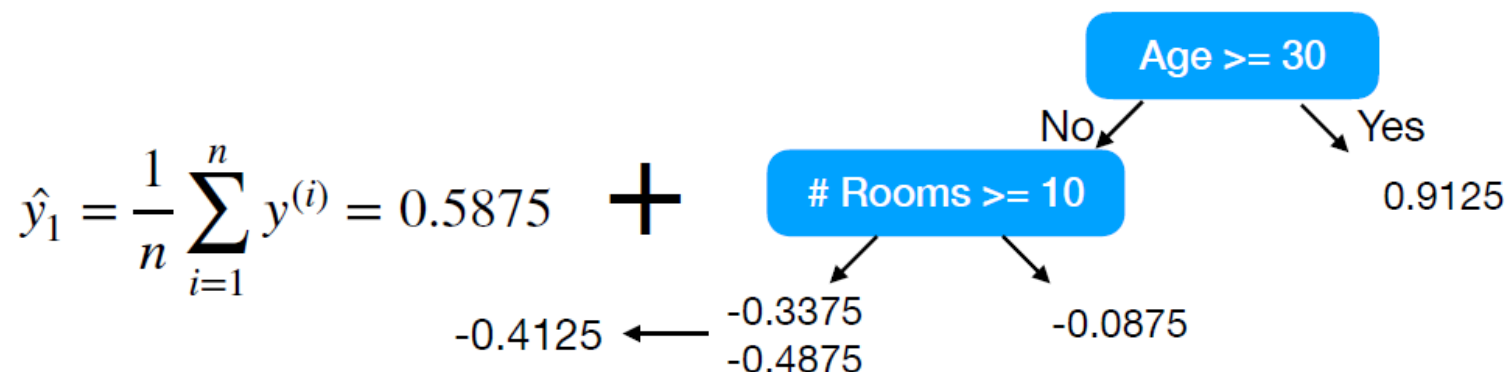


Gradient Boosting -- Conceptual Overview

--> A Regression-based Example

- **Step 3:** Combine tree from step 1 with trees from step 2

x1#	x2=City	x3=Age	y=Price	r=Res
5	Boston	30	1.5	$1.5 - 0.5875 = 0.9125$
10	Madison	20	0.5	$0.5 - 0.5875 = -0.0875$
6	Lansing	20	0.25	$0.25 - 0.5875 = -0.3375$
5	Waunake	10	0.1	$0.1 - 0.5875 = -0.4875$



Gradient Boosting -- Conceptual Overview

--> A Regression-based Example

- **Step 3:** Combine tree from step 1 with trees from step 2

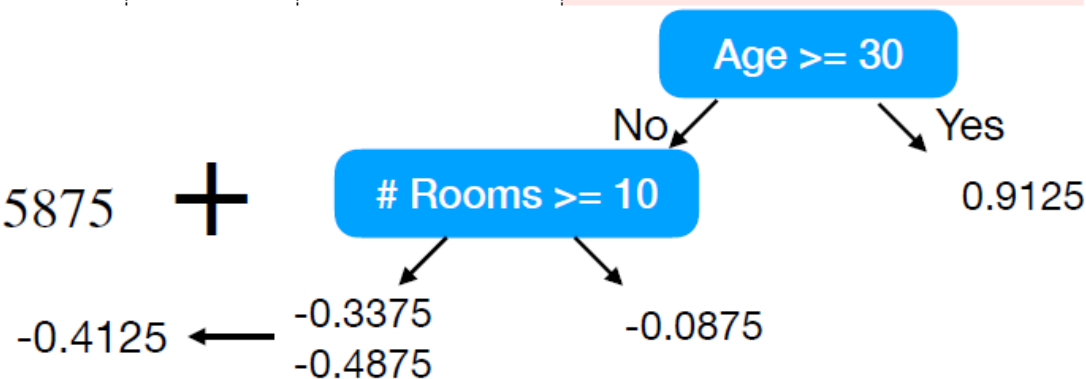
E.g.,
predict
Lansing



x1#	x2=City	x3=Age	y=Price	r=Res
5	Boston	30	1.5	$1.5 - 0.5875 = 0.9125$
10	Madison	20	0.5	$0.5 - 0.5875 = -0.0875$
6	Lansing	20	0.25	$0.25 - 0.5875 = -0.3375$
5	Waunakee	10	0.1	$0.1 - 0.5875 = -0.4875$

$$\hat{y}_1 = \frac{1}{n} \sum_{i=1}^n y^{(i)} = 0.5875$$

+



E.g.,
predict
Lansing

$$0.5875 + \alpha \times (-0.4125)$$

where α learning rate between 0 and 1 (if $\alpha = 1$, low bias but high variance)

Gradient Boosting -- Algorithm Overview

Step 0: Input data $\{ \langle \mathbf{x}^{(i)}, y^{(i)} \rangle \}_{i=1}^n$

Differentiable Loss function $L(y^{(i)}, h(\mathbf{x}^{(i)}))$

Step 1: Initialize model $h_0(\mathbf{x}) = \underset{\hat{y}}{\operatorname{argmin}} \sum_{i=1}^n L(y^{(i)}, \hat{y})$

Step 2: for $t = 1$ to T

A. Compute pseudo residual $r_{i,t} = - \left[\frac{\partial L(y^{(i)}, h(\mathbf{x}^{(i)}))}{\partial h(\mathbf{x}^{(i)})} \right]_{h(\mathbf{x})=h_{t-1}(\mathbf{x})}$
for $i = 1$ to n

B. Fit tree to $r_{i,t}$ values, and create
terminal nodes $R_{j,t}$ for $j = 1, \dots, J_t$

■ ■ ■

Gradient Boosting -- Algorithm Overview

Step 2: for $t = 1$ to T

A. Compute pseudo residual $r_{i,t} = - \left[\frac{\partial L(y^{(i)}, h(\mathbf{x}^{(i)}))}{\partial h(\mathbf{x}^{(i)})} \right]_{h(\mathbf{x})=h_{t-1}(\mathbf{x})}$
for $i = 1$ to n

B. Fit tree to $r_{i,t}$ values, and create terminal nodes $R_{j,t}$ for $j = 1, \dots, J_t$

C. for $j = 1, \dots, J_t$, compute

$$\hat{y}_{j,t} = \operatorname{argmin}_{\hat{y}} \sum_{\mathbf{x}^{(i)} \in R_{j,t}} L(y^{(i)}, h_{t-1}(\mathbf{x}^{(i)}) + \hat{y})$$

D. Update $h_t(\mathbf{x}) = h_{t-1}(\mathbf{x}) + \alpha \sum_{j=1}^{J_t} \hat{y}_{j,t} \mathbb{I}(\mathbf{x} \in R_{j,t})$

Step 3: Return $h_t(\mathbf{x})$

Gradient Boosting -- Algorithm Overview Discussion

Step 0: Input data $\{ \langle \mathbf{x}^{(i)}, y^{(i)} \rangle \}_{i=1}^n$

Differentiable Loss function $L(y^{(i)}, h(\mathbf{x}^{(i)}))$



E.g., Sum-squared error in regression

$$SSE' = \frac{1}{2} (y^{(i)} - h(\mathbf{x}^{(i)}))^2$$

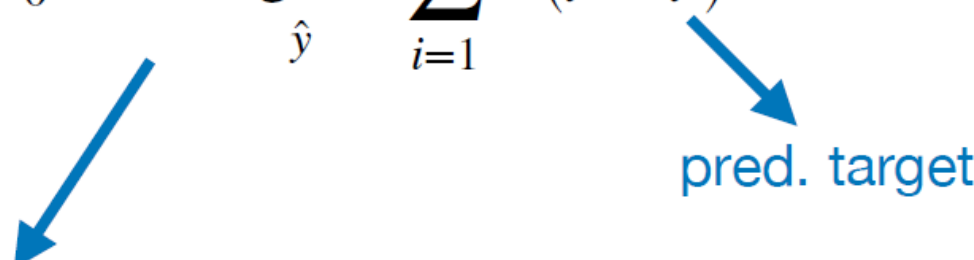
$$\frac{\partial}{\partial h(\mathbf{x}^{(i)})} \frac{1}{2} (y^{(i)} - h(\mathbf{x}^{(i)}))^2 \quad [\text{chain rule}]$$

$$= 2 \times \frac{1}{2} (y^{(i)} - h(\mathbf{x}^{(i)})) \times (0 - 1) = - (y^{(i)} - h(\mathbf{x}^{(i)}))$$

[neg. residual]

Gradient Boosting -- Algorithm Overview Discussion

Step 1: Initialize model $h_0(\mathbf{x}) = \underset{\hat{y}}{\operatorname{argmin}} \sum_{i=1}^n L(y^{(i)}, \hat{y})$



turns out to be the average (in regression)

pred. target

$$\frac{1}{n} \sum_{i=1}^n y^{(i)}$$

Gradient Boosting -- Algorithm Overview Discussion

Loop to make T trees (e.g., $T=100$)

Step 2: for $t = 1$ to T

A. Compute pseudo residual $r_{i,t} = - \left[\frac{\partial L(y^{(i)}, h(\mathbf{x}^{(i)}))}{\partial h(\mathbf{x}^{(i)})} \right]_{h(\mathbf{x})=h_{t-1}(\mathbf{x})}$ for $i = 1$ to n

pseudo residual of the t -th tree and i -th example

Derivative of the loss function

Gradient Boosting -- Algorithm Overview Discussion

Loop to make T trees (e.g., $T=100$)

Step 2: for $t = 1$ to T

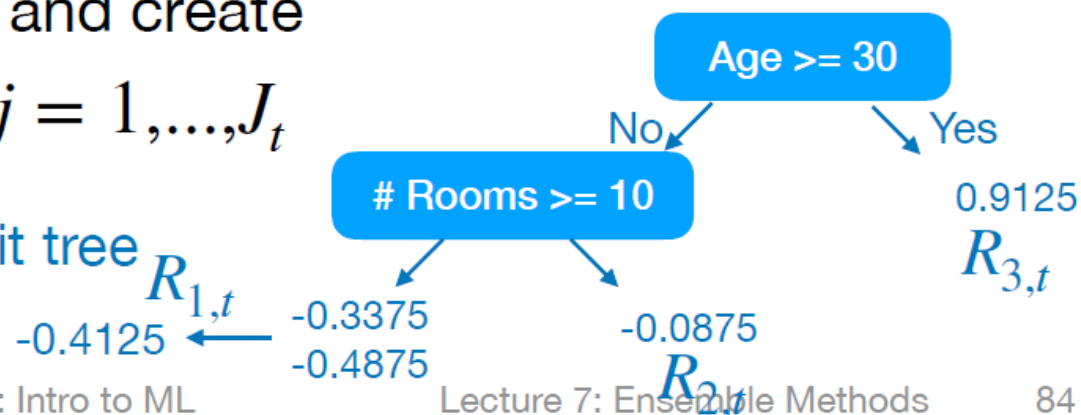
A. Compute pseudo residual $r_{i,t} = - \left[\frac{\partial L(y^{(i)}, h(\mathbf{x}^{(i)}))}{\partial h(\mathbf{x}^{(i)})} \right]_{h(\mathbf{x})=h_{t-1}(\mathbf{x})}$ for $i = 1$ to n

pseudo residual of the t -th tree and i -th example

Derivative of the loss function

B. Fit tree to $r_{i,t}$ values, and create terminal nodes $R_{j,t}$ for $j = 1, \dots, J_t$

Use features in dataset to fit tree



Gradient Boosting -- Algorithm Overview Discussion

Step 2: for $t = 1$ to T

A. Compute pseudo residual $r_{i,t} = - \left[\frac{\partial L(y^{(i)}, h(\mathbf{x}^{(i)}))}{\partial h(\mathbf{x}^{(i)})} \right]_{h(\mathbf{x})=h_{t-1}(\mathbf{x})}$
for $i = 1$ to n

B. Fit tree to $r_{i,t}$ values, and create terminal nodes $R_{j,t}$ for $j = 1, \dots, J_t$

C. for $j = 1, \dots, J_t$, compute

$$\hat{y}_{j,t} = \operatorname{argmin}_{\hat{y}} \sum_{\mathbf{x}^{(i)} \in R_{i,j}} L(y^{(i)}, h_{t-1}(\mathbf{x}^{(i)}) + \hat{y})$$

Compute the residual for each leaf node

Only consider examples at that leaf node

Like step 1 but add previous prediction

Gradient Boosting -- Algorithm Overview Discussion

Step 2: for $t = 1$ to T

A. Compute pseudo residual $r_{i,t} = - \left[\frac{\partial L(y^{(i)}, h(\mathbf{x}^{(i)}))}{\partial h(\mathbf{x}^{(i)})} \right]_{h(\mathbf{x})=h_{t-1}(\mathbf{x})}$
for $i = 1$ to n

B. Fit tree to $r_{i,t}$ values, and create terminal nodes $R_{j,t}$ for $j = 1, \dots, J_t$

C. for $j = 1, \dots, J_t$, compute

$$\hat{y}_{j,t} = \operatorname{argmin}_{\hat{y}} \sum_{\mathbf{x}^{(i)} \in R_{j,t}} L(y^{(i)}, h_{t-1}(\mathbf{x}^{(i)}) + \hat{y})$$

D. Update $h_t(\mathbf{x}) = h_{t-1}(\mathbf{x}) + \alpha \sum_{j=1}^{J_t} \hat{y}_{j,t} \mathbb{I}(\mathbf{x} \in R_{j,t})$

learning rate
between 0 and 1
(usually 0.1)

Summation just in case
examples end up in
multiple nodes

Gradient Boosting -- Algorithm Overview Discussion

For prediction, combine all T trees, e.g.,

$$h_0(\mathbf{x}) = \operatorname{argmin}_{\hat{y}} \sum_{i=1}^n L(y^{(i)}, \hat{y})$$

$$+ \alpha \hat{y}_{j,t=1} = \operatorname{argmin}_{\hat{y}} \sum_{\mathbf{x}^{(i)} \in R_{i,j}} L(y^{(i)}, h_{(t=1)-1}(\mathbf{x}^{(i)}) + \hat{y})$$

...

$$+ \alpha \hat{y}_{j,T} = \operatorname{argmin}_{\hat{y}} \sum_{\mathbf{x}^{(i)} \in R_{i,j}} L(y^{(i)}, h_{T-1}(\mathbf{x}^{(i)}) + \hat{y})$$

Gradient Boosting -- Algorithm Overview Discussion

For prediction, combine all T trees, e.g.,

$$h_0(\mathbf{x}) = \operatorname{argmin}_{\hat{y}} \sum_{i=1}^n L(y^{(i)}, \hat{y})$$

$$+ \alpha \hat{y}_{j,t=1}$$

...

$$+ \alpha \hat{y}_{j,T}$$

The idea is that we decrease the
pseudo residuals by a small amount
at each step

XGBoost

Summary and Main Points:

- scalable implementation of gradient boosting
- Improvements include: regularized loss, sparsity-aware algorithm, weighted quantile sketch for approximate tree learning, caching of access patterns, data compression, sharding
- Decision trees based on CART
- Regularization term for penalizing model (tree) complexity
- Uses second order approximation for optimizing the objective
- Options for column-based and row-based subsampling
- Single-machine version of XGBoost supports the exact greedy algorithm

Chen, T., & Guestrin, C. (2016, August). XGBoost: A scalable tree boosting system. In *Proceedings of the 22nd International Conference on Knowledge Discovery and Data Mining* (pp. 785-794). ACM.



Speed

We compared speed using only the training task without any test or metric output. We didn't count the time for IO. For the ranking tasks, since XGBoost and LightGBM implement different ranking objective functions, we used `regression` objective for speed benchmark, for the fair comparison.

The following table is the comparison of time cost:

Data	xgboost	xgboost_hist	LightGBM
Higgs	3794.34 s	165.575 s	130.094 s
Yahoo LTR	674.322 s	131.462 s	76.229 s
MS LTR	1251.27 s	98.386 s	70.417 s
Expo	1607.35 s	137.65 s	62.607 s
Allstate	2867.22 s	315.256 s	148.231 s

LightGBM ran faster than xgboost on all experiment data sets.

Accuracy

We computed all accuracy metrics only on the test data set.

Data	Metric	xgboost	xgboost_hist	LightGBM
Higgs	AUC	0.839593	0.845314	0.845724
Yahoo LTR	NDCG ₁	0.719748	0.720049	0.732981
	NDCG ₃	0.717813	0.722573	0.735689
	NDCG ₅	0.737849	0.740899	0.75352
	NDCG ₁₀	0.78089	0.782957	0.793498
MS LTR	NDCG ₁	0.483956	0.485115	0.517767
	NDCG ₃	0.467951	0.47313	0.501063
	NDCG ₅	0.472476	0.476375	0.504648
	NDCG ₁₀	0.492429	0.496553	0.524252
Expo	AUC	0.756713	0.776224	0.776935
Allstate	AUC	0.607201	0.609465	0.609072

<https://lightgbm.readthedocs.io/en/latest/Experiments.html>

Code 1

```
from sklearn import model_selection
from sklearn.tree import DecisionTreeClassifier
from sklearn.model_selection import train_test_split
from sklearn import datasets

iris = datasets.load_iris()
X, y = iris.data[:, [0, 3]], iris.target

X_train, X_test, y_train, y_test = \
    train_test_split(X, y, test_size=0.25, random_state=1)

X_train, X_val, y_train, y_val = \
    train_test_split(X_train, y_train, test_size=0.25, random_state=1)

print('Train/Valid/Test sizes:', y_train.shape[0], y_val.shape[0], y_test.shape[0])
```

Train/Valid/Test sizes: 84 28 38

```
from sklearn.ensemble import GradientBoostingClassifier

boost = GradientBoostingClassifier(
    learning_rate=0.1,
    n_estimators=100,
    max_depth=8,
    random_state=1)

boost.fit(X_train, y_train)

print("Test Accuracy: %0.2f" % boost.score(X_test, y_test))
```

Test Accuracy: 0.95

Code 2

```
from sklearn.experimental import enable_hist_gradient_boosting
from sklearn.ensemble import HistGradientBoostingClassifier

boost = HistGradientBoostingClassifier(
    learning_rate=0.1,
    #n_estimators=100,
    max_depth=8,
    random_state=1)

boost.fit(X_train, y_train)

print("Test Accuracy: %0.2f" % boost.score(X_test, y_test))
```

Test Accuracy: 0.97

Code 3

```
# https://xgboost.readthedocs.io/en/latest/build.html
```

```
#!/pip install xgboost
```

```
import numpy as np
import xgboost as xgb

dtrain = xgb.DMatrix(X_train, label=y_train)
dtest = xgb.DMatrix(X_test, label=y_test)

param = {
    'max_depth': 8,
    'eta': 0.1, # learning rate
    'objective': 'multi:softprob', # loss function for multiclass
    'num_class': 3} # number of classes

boost = xgb.train(param, dtrain, num_boost_round=100)

y_pred = boost.predict(dtest)
y_labels = np.argmax(y_pred, axis=1)

print("Test Accuracy: %0.2f" % (y_labels == y_test).mean())
```

```
Test Accuracy: 0.97
```

Code 4

```
# https://lightgbm.readthedocs.io/en/latest/Installation-Guide.html
# conda install -c conda-forge lightgbm

import lightgbm as lgb

boost = lgb.LGBMClassifier(n_estimators=100,
                           max_depth=8,
                           random_state=1,
                           learning_rate=0.1)

boost.fit(X_train, y_train)

print("Test Accuracy: %0.2f" % boost.score(X_test, y_test))

Test Accuracy: 1.00
```




Random forests

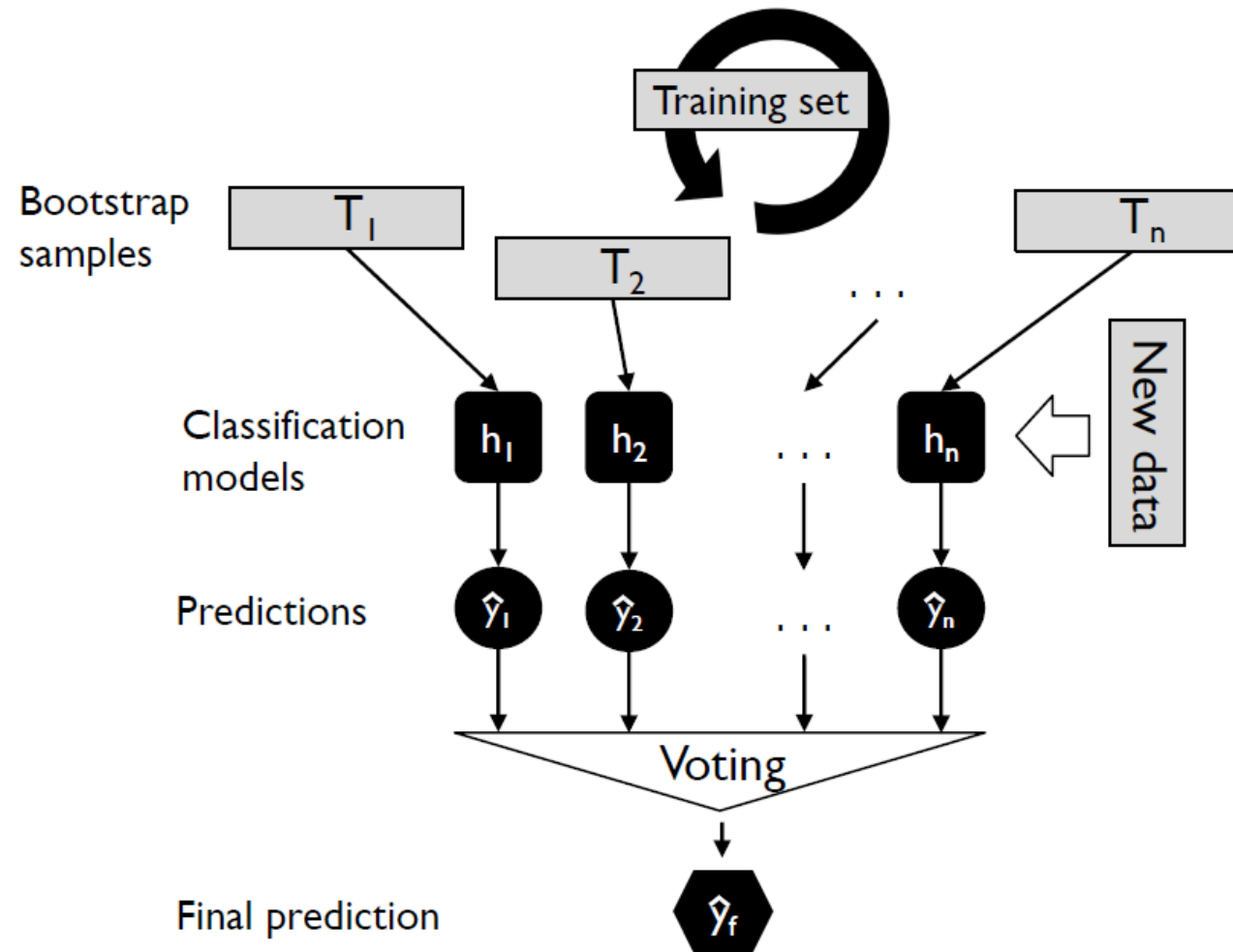
Bagging w. trees + random feature subsets

<https://youtu.be/r5C3TUIw6Zk?si=2y65TBCgaw2EsXNv>

Random forest algorithm

1. Draw a random **bootstrap** sample of size n (randomly choose n examples from the training dataset with replacement).
2. Grow a decision tree from the bootstrap sample. At each node:
 - a. Randomly select d features without replacement.
 - b. Split the node using the feature that provides the best split according to the objective function, for instance, maximizing the information gain.
3. Repeat *steps 1-2* k times.
4. Aggregate the prediction by each tree to assign the class label by **majority vote**

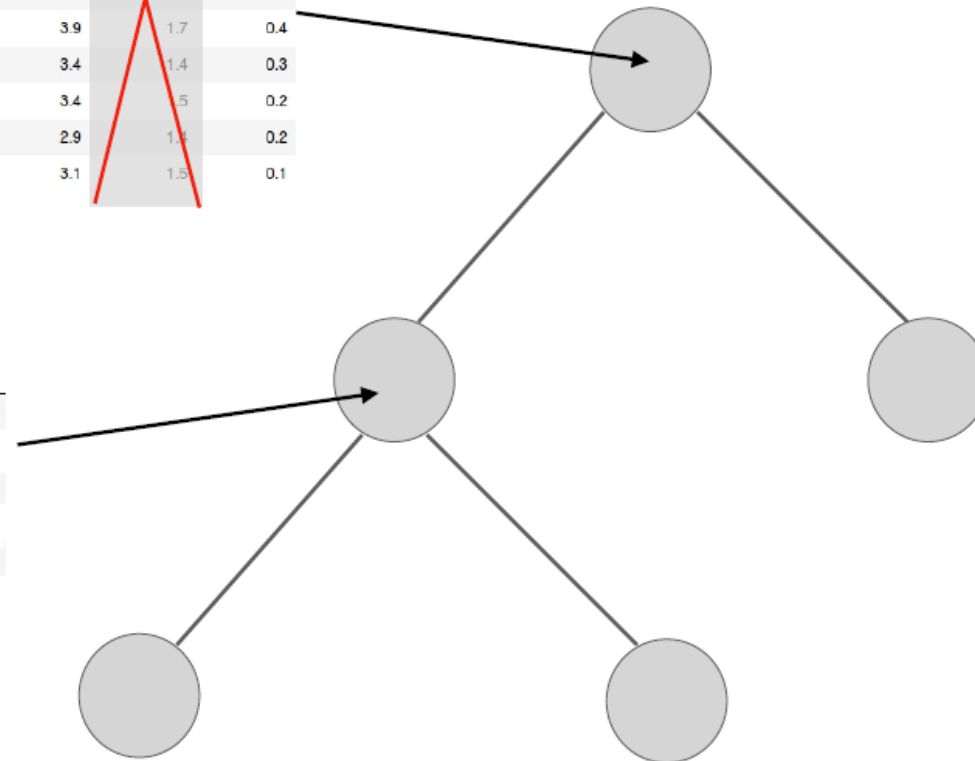
Bagging Classifier

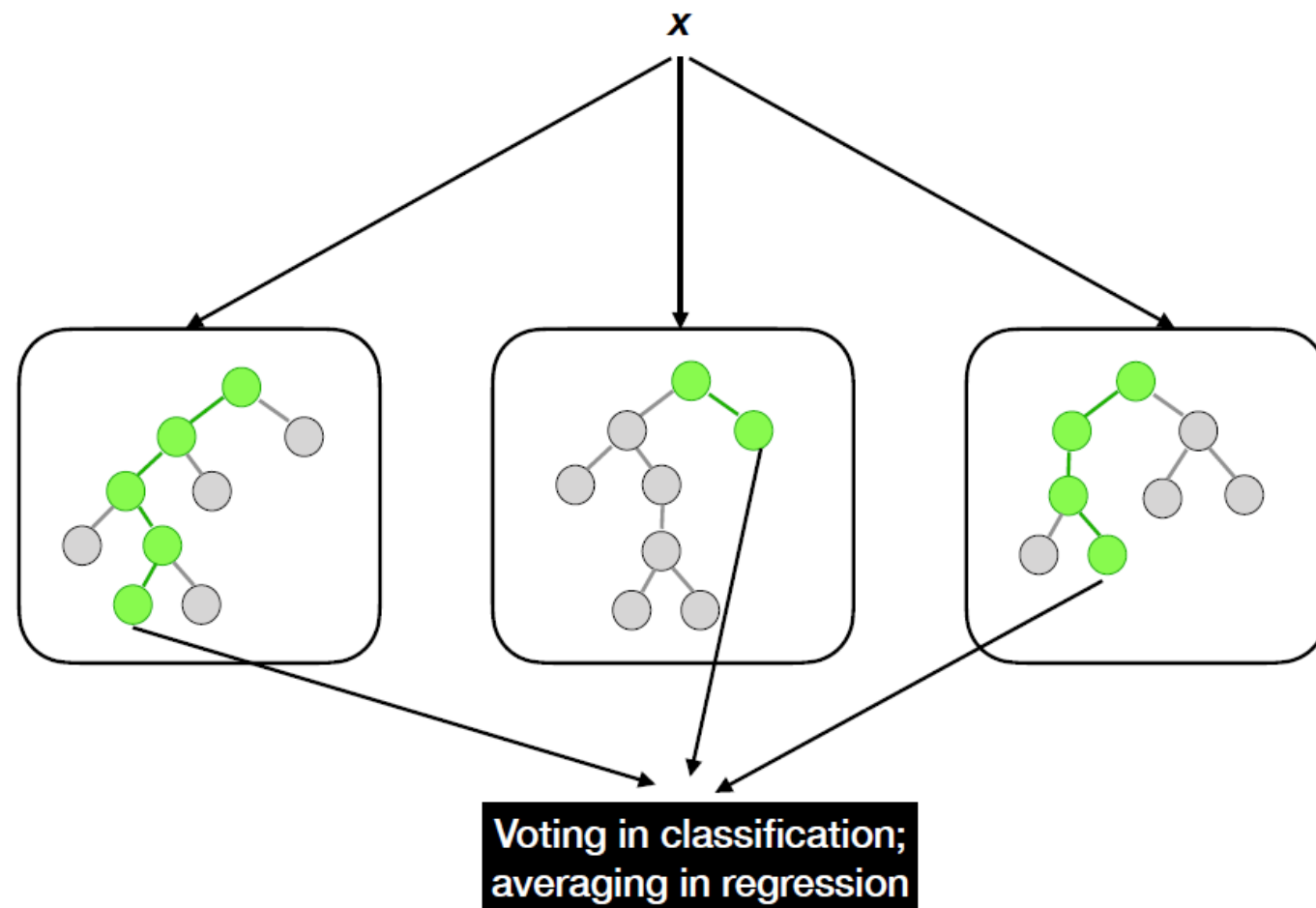


Random Feature Subsets

	sepal_length	sepal_width	petal_length	petal_width
0	5.1	3.5	1.4	0.2
1	4.8	3.0	1.4	0.2
2	4.7	3.2	1.3	0.2
3	4.6	3.1	1.5	0.2
4	5.0	3.6	1.4	0.2
5	5.4	3.9	1.7	0.4
6	4.6	3.4	1.4	0.3
7	5.0	3.4	1.5	0.2
8	4.4	2.9	1.4	0.2
9	4.9	3.1	1.5	0.1

	sepal_length	sepal_width	petal_length	petal_width
0	5.1	3.5	1.4	0.2
2	4.7	3.2	1.3	0.2
4	5.0	3.6	1.4	0.2
5	5.4	3.9	1.7	0.4
9	4.9	3.1	1.5	0.1





In contrast to the original publication
[Breiman, “Random Forests”, Machine Learning, 45(1), 5-32, 2001]
the scikit-learn implementation combines classifiers by
averaging their probabilistic prediction, instead of letting each
classifier vote for a single class.

"Soft Voting"

"Soft" Voting

$$\hat{y} = \arg \max_j \sum_{i=1}^n w_i p_{i,j}$$

$p_{i,j}$: predicted class membership
probability of the i th classifier for
class label j

w_i : optional weighting parameter, default
 $w_i = 1/n, \forall w_i \in \{w_1, \dots, w_n\}$

"Soft" Voting

$$\hat{y} = \arg \max_j \sum_{i=1}^n w_i p_{i,j}$$

Binary classification example

$$j \in \{0,1\} \quad h_i(i \in \{1,2,3\})$$

$$h_1(\mathbf{x}) \rightarrow [0.9, 0.1]$$

$$h_2(\mathbf{x}) \rightarrow [0.8, 0.2]$$

$$h_3(\mathbf{x}) \rightarrow [0.4, 0.6]$$

$$p(j = 0 | \mathbf{x}) = 0.2 \cdot 0.9 + 0.2 \cdot 0.8 + 0.6 \cdot 0.4 = 0.58$$

$$p(j = 1 | \mathbf{x}) = 0.2 \cdot 0.1 + 0.2 \cdot 0.2 + 0.6 \cdot 0.6 = 0.42$$

$$\hat{y} = \arg \max_j \left\{ p(j = 0 | \mathbf{x}), p(j = 1 | \mathbf{x}) \right\}$$

Code

```
from sklearn import model_selection
from sklearn.model_selection import train_test_split
from sklearn import datasets

iris = datasets.load_iris()
X, y = iris.data[:, [0, 3]], iris.target

X_train, X_test, y_train, y_test = \
    train_test_split(X, y, test_size=0.25, random_state=1)

X_train, X_val, y_train, y_val = \
    train_test_split(X_train, y_train, test_size=0.25, random_state=1)

print('Train/Valid/Test sizes:', y_train.shape[0], y_val.shape[0], y_test.shape[0])
```

Train/Valid/Test sizes: 84 28 38

```
from sklearn.ensemble import RandomForestClassifier

forest = RandomForestClassifier(n_estimators=100,
                               random_state=1)

forest.fit(X_train, y_train)

print("Test Accuracy: %0.2f" % forest.score(X_test, y_test))
```

Test Accuracy: 0.95

```
from sklearn.ensemble import ExtraTreesClassifier

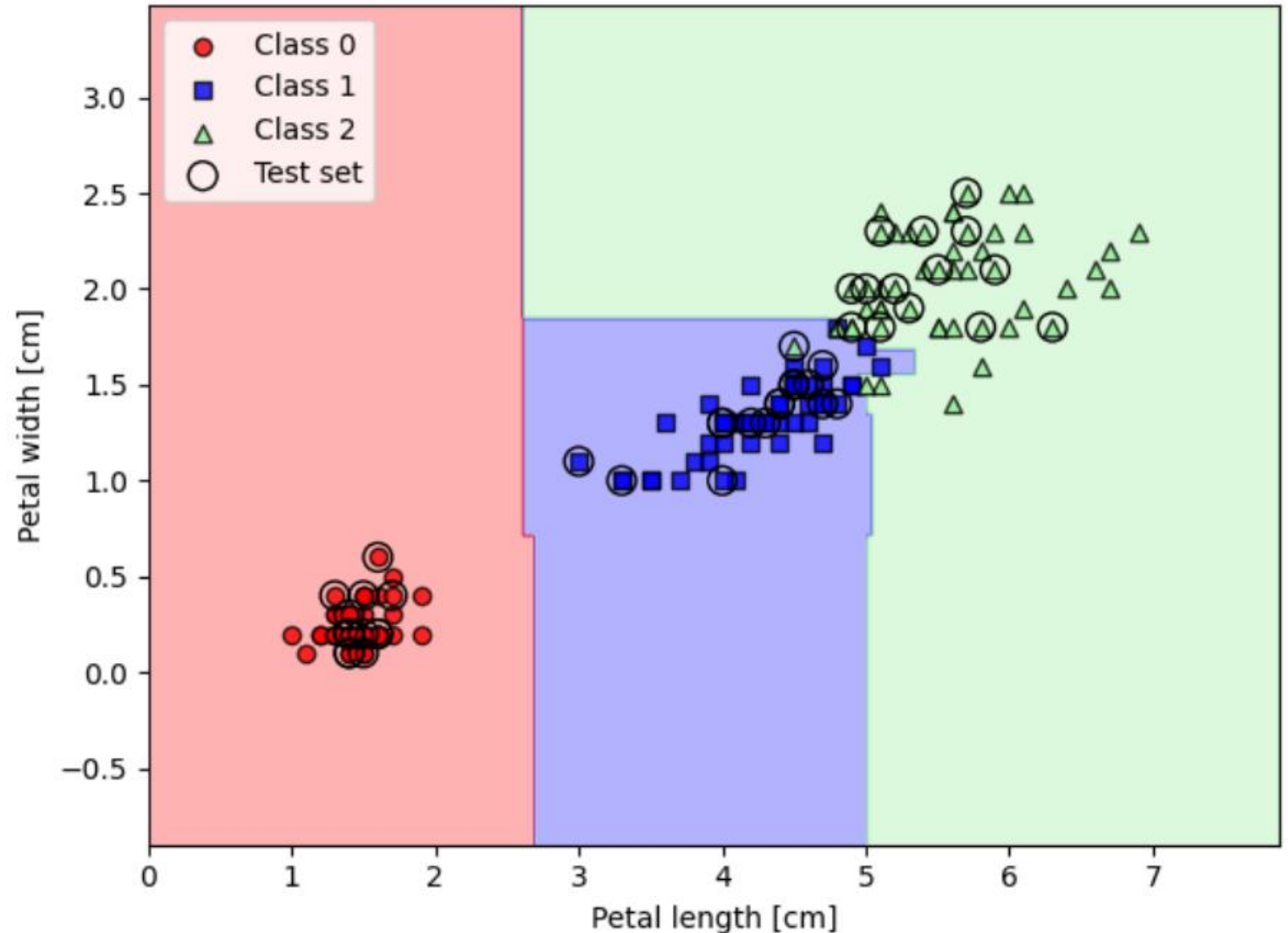
forest = ExtraTreesClassifier(n_estimators=100,
                              random_state=1)

forest.fit(X_train, y_train)

print("Test Accuracy: %0.2f" % forest.score(X_test, y_test))
```

Test Accuracy: 0.95

Classification of iris dataset with random forest



References



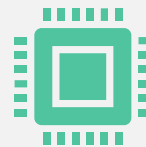
1. S. Raschka, Y. Liu and V. Mirjalili. Machine Learning with PyTorch and Scikit-Learn. Packt Publishing. 2022.

<https://sebastianraschka.com/blog/2022/ml-pytorch-book.html>



2. A. Ng. CS229 Lecture Notes. Stanford University. 2023.

https://cs229.stanford.edu/main_notes.pdf



3. T. Broderick. Introduction to Machine Learning course (6.036). Massachusetts Institute of Technology. 2020.

<https://tamarabroderick.com/ml.html>

4. S. Raschka. **Introduction to Machine Learning course.(STAT 451)** University of Wisconsin, Madison. 2021

Selected videos from youtube

- Sebastian Raschka, [Intro to machine learning](#), L07